НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Департамент программной инженерии

ОТЧЁТ

по индивидуальному заданию №3
по дисциплине
«Архитектура компьютера и операционные системы»

Вариант 29. Группа лифтов

Разработка имитационной программы
с использованием взаимодействующих процессов

Выполнил: Буров Иван Васильевич
Группа: БПИ252

Москва, 2026

1 Условие задачи

В многоэтажном здании работает группа лифтов под управлением общего диспетчера. Пассажиры появляются через случайные интервалы, получают уникальные номера и задают этажи назначения. Вызов содержит этаж отправления и направление. Диспетчер назначает вызовы лифтам по выбранной стратегии. Кабина не принимает людей сверх вместимости; оставшиеся продолжают ожидание, их вызовы не должны теряться.

Входные параметры: число этажей, лифтов и мест; начальное положение кабин; число пассажиров и пары этажей отправления и назначения; диапазон интервалов появления; время движения между соседними этажами, открытия и закрытия дверей, посадки и высадки; стратегия назначения.

Инварианты: вместимость не превышается; посадка и высадка возможны только при открытых дверях на этаже остановки; движение с открытыми дверями запрещено; пассажир не находится одновременно в двух местах; высадка происходит только на этаже назначения; один вызов не назначается двум лифтам без разделения пассажиров.

Условие допускает завершение после доставки всех пассажиров либо после заданного периода работы. В этой реализации выбран первый вариант. Дополнительно поддерживается остановка по SIGINT и SIGTERM с выводом состояний оставшихся пассажиров.

2 Отображение сущностей в процессы

Диспетчер, каждый лифт и каждый пассажир представлены отдельными процессами. При L лифтах и P пассажирах модель использует $1 + L + P$ процессов. Отдельный процесс генератора не требуется: интервалы вычисляются заранее, а каждый пассажир самостоятельно ждёт своего времени появления.

Сущность	**Число**	**Поведение процесса**
Диспетчер	1	хранит модель, назначает вызовы, обрабатывает события, пишет журнал и статистику
Лифт	L	выполняет команды движения и операций с дверями, посылает подтверждения
Пассажир	P	появляется в заданное время, сообщает о вызове, выполняет посадку и высадку по командам

У диспетчера находятся обычные массивы `Passenger` и `Lift`. Другие процессы не изменяют их напрямую. Они передают сообщения, и только диспетчер меняет подтверждённое состояние модели. Поэтому для этих массивов не нужен общий мьютекс.

Общего такта нет. Участник выдерживает задержку через `nanosleep`, пока другие процессы могут выполнять свои действия. Для реакции на остановку длительная пауза разбита на отрезки до 100 мс. Это не задаёт частоту моделирования и не синхронизирует движение кабин между собой.

Жизненный цикл пассажира:

```
FUTURE -> WAITING -> RESERVED -> RIDING -> DELIVERED
```

`RESERVED` означает, что пассажиру назначены лифт и место, но посадка ещё не завершена. После подтверждения посадки состояние становится `RIDING`, после подтверждения высадки — `DELIVERED`.

Лифту назначается группа уже ожидающих пассажиров с одного этажа и одного направления, не больше вместимости. Остальные остаются в `WAITING` и сохраняют свои индивидуальные вызовы. Группа фиксируется при назначении: новые попутные вызовы в текущий рейс не добавляются. После посадки кабина последовательно обслуживает этажи назначения в выбранном направлении. Новое направление назначается после высадки всей группы и закрытия дверей.

Реализованы две стратегии: `strategy=0` выбирает старейший зарегистрированный вызов, затем ближайший свободный лифт; `strategy=1` выбирает ближайшую пару «свободный лифт — ожидающий пассажир», при равных расстояниях учитывая порядок регистрации. Подбор остальных пассажиров группы не меняет принадлежность уже назначенных вызовов.

3 Первые две реализации: общий родитель и общая память

Программы `elevator-unnamed` и `elevator-named` создают детей через `fork`. Родитель одновременно является диспетчером. До создания детей он подготавливает память и семафоры, поэтому дети наследуют готовые отображения и открытые объекты.

Для каждого участника существует входящая ячейка `Mailbox`. Отдельная общая ячейка предназначена для событий диспетчеру. Всего ячеек $N = 1 + L + P$. Адресат команды определяется номером ячейки; в сообщении события указан номер отправителя.

В каждой ячейке два семафора. Начальное значение `can_write` равно единице, `can_read` — нулю:

```
отправитель: wait(can_write) -> запись -> post(can_read)
получатель:  wait(can_read)  -> копия  -> post(can_write)
```

Семафор записи не позволяет двум отправителям одновременно заполнять общую ячейку событий. Получатель копирует сообщение до разрешения новой записи. Следующая команда конкретному исполнителю выдаётся после подтверждения предыдущего действия, поэтому его входящая ячейка не переполняется командами.

Первая реализация. Объект создаётся через `shm_open`, размер задаётся `ftruncate`, затем вызывается `mmap` с `MAP_SHARED`. Неименованные семафоры находятся внутри отображения и инициализируются `sem_init` с `pshared=1`. После завершения детей выполняются `sem_destroy`, `munmap` и `shm_unlink`.

Вторая реализация. Сообщения по-прежнему находятся в общей памяти POSIX, а семафоры создаются через `sem_open` с `O_CREAT | O_EXCL`. Имена включают PID диспетчера и индекс. Массив указателей на открытые семафоры находится в обычной памяти; дети наследуют его при `fork`. Освобождение выполняется через `sem_close` и `sem_unlink`.

Обе версии собраны из общих исходников. Условная компиляция выбирает способ создания и получения семафоров, сохраняя одинаковый протокол сообщений и модель.

4 Третья реализация: независимый запуск

`elevator-server` и `elevator-client` запускаются отдельными командами. Клиент с номером $1 \dots L$ выполняет роль лифта; с номером $L+1 \dots L+P$ — пассажира. Диспетчер не вызывает `fork` для создания этих процессов. Клиенты читают тот же входной файл и открывают объекты сеанса по имени.

Сервер создаёт общую память POSIX с параметрами сеанса, временем старта и ячейками сообщений, затем именованные семафоры. Последним публикуется семафор готовности. Клиенты запускаются после сообщения «Готово».

Регистрация защищена семафором: два процесса не могут занять один номер. Клиент проверяет параметры, занимает номер и посылает `REGISTER` со своим PID. Диспетчер ждёт все $L + P$ регистрации, записывает общее начало отсчёта и выдаёт `START` каждому участнику. После этого начинается прежний обмен командами и событиями.

Так задержка ручного запуска не считается ожиданием лифта и не меняет заданные интервалы появления пассажиров. Отдельный процесс получает свой адрес отображения через `mmap`; указатели из адресного пространства сервера ему не передаются.

5 Четвёртая реализация: очереди сообщений POSIX

`elevator-mq-server` и `elevator-mq-client` также запускаются независимо. Вместо отображаемых ячеек создаются именованные очереди POSIX через `mq_open`. Каждая очередь рассчитана на одно сообщение размера `Message`. Отправка и получение используют `mq_send` и `mq_receive`; пара именованных семафоров сохраняет протокол свободной и заполненной ячейки.

В этой сборке `Session` размещается через `calloc` и является частной памятью процесса. Вызовов `shm_open` и отображения общей памяти нет. Клиент передаёт параметры внутри `REGISTER`, а время старта получает в `START`.

Для ответа на регистрацию клиент создаёт дополнительную очередь с PID в имени. Сервер отвечает именно туда: либо принимает номер, либо посылает `REJECT`. Это важно для повторного подключения: отказ не должен получить настоящий участник, уже читающий очередь данного номера. После отправки ответа сервер удаляет имя временной очереди; клиент может забрать сообщение через уже открытый дескриптор и закрывает его.

Основные очереди открыты неблокирующим способом, поскольку наличие сообщения и свободного места уже согласовано семафорами. Ошибки `mq_send` и `mq_receive` проверяются. Диспетчер закрывает и удаляет созданные очереди при завершении.

6 Проверка инвариантов

Корректность обеспечивается состояниями модели и проверками при обработке подтверждений, а не отдельным потоком наблюдения.

Инвариант	**Как обеспечивается**
Вместимость	при назначении резервируется не больше мест; при подтверждении посадки проверяется текущая загрузка

Инвариант	**Как обеспечивается (продолжение)**
Посадка и высадка	проверяются открытые двери, фаза передачи пассажиров и соответствующий этаж
Закрытые двери при движении	команда движения и её подтверждение проверяют состояние дверей
Одно положение пассажира	один элемент массива имеет одно состояние и один номер назначенного лифта; переходы выполняет диспетчер
Правильная высадка	этаж кабины сравнивается с этажом назначения
Единственное назначение	назначаются только пассажиры в `WAITING`, затем они переходят в `RESERVED`

Неподходящее подтверждение считается ошибкой, приводит к остановке сеанса и освобождению ресурсов. При обычном завершении каждый пассажир доставляется ровно один раз. Дополнительно эти свойства проверяются по результатам тестовых запусков.

7 Пользовательский интерфейс и входные данные

Программа является консольной. Для первых двух версий интерфейс:

```
./build/elevator-unnamed INPUT [LOG]
./build/elevator-named INPUT [LOG]
```

Для независимых версий интерфейс:

```
./build/elevator-server INPUT SESSION [LOG]
./build/elevator-client INPUT SESSION ACTOR
./build/elevator-mq-server INPUT SESSION [LOG]
./build/elevator-mq-client INPUT SESSION ACTOR
```

`SESSION` — общее имя запуска, `ACTOR` — номер участника. Если путь журнала не указан, используется `elevator-03.log`. Все команды выполняются из каталога `IHW-3`.

Числа во входном файле идут в следующем порядке:

```
floors lifts capacity passengers seed strategy
arrival_min arrival_max travel_ms door_ms board_ms leave_ms
начальные этажи всех лифтов
пары from to для каждого пассажира
```

Все длительности измеряются в миллисекундах. `travel_ms` задаёт время на один этаж, `door_ms` — время открытия и отдельно закрытия дверей, `board_ms` и `leave_ms` — время одного пассажира. Интервалы появления накапливаются последовательно.

Например, файл `basic.in`:

```
5 2 2 4 42 0
50 100 150 100 50 50
1 5
1 5
5 1
2 4
4 2
```

Это пять этажей, два лифта по два места и четыре пассажира; начальные этажи кабин — 1 и 5. Используется стратегия 0 и начальное значение генератора 42.

Проверяются границы: этажей не меньше двух, лифтов и мест не меньше одного; пассажиров может быть ноль; этажи принадлежат зданию, начало и конец маршрута различаются. Все значения неотрицательны и помещаются в `int`. Проверяются порядок границ интервалов, стратегия, переполнения размеров и лишние данные. Ошибка выявляется до запуска модели. Ввод использует `open` и `read`.

Ниже пример независимого запуска для `basic.in`:

```
./build/elevator-server basic.in demo independent.log &
server_pid=$!
# Дождаться строки «Готово», затем:
for actor in 1 2 3 4 5 6; do
    ./build/elevator-client basic.in demo "$actor" &
done
wait "$server_pid"
echo $?
wait
```

Для четвёртой реализации используются программы с префиксом `elevator-mq-` и другое имя сеанса. Все участники должны работать в одной ОС и пространстве IPC, при использовании Docker — в одном контейнере.

Вывод. Диспетчер печатает временную отметку и событие: появление пассажира, регистрацию и назначение вызова, начало движения, прибытие, открытие и закрытие дверей, посадку, высадку и доставку. Строки не перемешиваются, поскольку участники сами журнал не пишут. Те же события через файловый дескриптор записываются в журнал функцией `dprintf`.

8 Завершение и освобождение ресурсов

Штатное завершение наступает после доставки всех пассажиров и закрытия дверей кабин. Лифты и уже доставленные пассажиры получают `STOP`. Нулевое число пассажиров также корректно завершает сеанс.

Обработчики SIGINT и SIGTERM только записывают номер сигнала в переменную типа `volatile sig_atomic_t`. Печать, ожидание и удаление ресурсов выполняются обычным кодом. Семафоры ожидаются через `sem_timedwait` с короткими интервалами, чтобы повторно проверять остановку и состояние процессов.

В версиях с `fork` родитель останавливает детей и собирает их статусы через `waitpid`. После этого можно уничтожать неименованные семафоры. В независимых версиях

`waitpid` неприменим к участникам, которые не являются детьми диспетчера. Окончание роли подтверждается сообщением `BYE`; при ошибке зарегистрированным участникам посылается SIGTERM. Ожидание подтверждений при завершении ограничено пятью секундами.

Ресурсы освобождаются и при частичной ошибке создания: учитывается, сколько семафоров и очередей уже открыто. Удаляются только имена, созданные данным диспетчером. Клиенты закрывают свои обращения, но не удаляют общие объекты работающего сервера.

Код	**Значение для диспетчера**
0	штатное завершение
1	ошибка данных, системного вызова или протокола
2	неверное число аргументов командной строки
130	остановка по SIGINT
143	остановка по SIGTERM

В конце выводятся число доставленных пассажиров и среднее ожидание завершения посадки. Делитель — число завершивших посадку, включая ещё находящихся в кабине, а не только доставленных. Для недоставленных печатаются последние подтверждённые состояния; номер лифта 0 означает, что назначения ещё нет.

SIGKILL, авария ядра и отключение питания не позволяют выполнить обработчик и очистку. Автоматическое удаление имён в этих случаях не гарантируется. Время остановки также зависит от планировщика ОС.

9 Тестирование и результаты

Проверка выполнена в Linux через Docker, образ `gcc:14`: ядро `7.0.14-linuxkit`, архитектура `aarch64`, GCC 14.4.0, Python 3.13.5. Все шесть исполняемых файлов пересобраны с флагами:

```
-std=c11 -Wall -Wextra -Wpedantic -Werror -O2
```

Сборка завершилась без предупреждений и ошибок. Проверяемые реализации используют C11, libc и POSIX API. На macOS программа запускается в Linux-контейнере, а не как нативное приложение.

Воспроизводимый основной прогон:

```
make -B check \
    CFLAGS='-std=c11 -Wall -Wextra -Wpedantic -Werror -O2'
```

`test.py` запускает шесть сценариев на каждой из четырёх реализаций — всего 24 проверки. Повторный прогон перед подготовкой отчёта завершился строкой «Все 24 проверки пройдены».

Набор	**Что проверено**	**Итог**
`basic.in`	два лифта, четыре пассажира, оба направления	4/4, код 0
`group.in`	группа, вместимость 2, несколько остановок	5/5, код 0
`nearest.in`	стратегия 1, движение вниз	3/3, код 0
`empty.in`	нет пассажиров	0/0, код 0
`invalid.in`	вместимость 0	отказ, код 1
`interrupt.in`	SIGINT во время движения	итог, код 130

Скрипт проверяет уникальность доставки, этажи назначения, загрузку, наличие итога в выводе и журнале, коды возврата и завершение запущенных процессов. После каждого сценария проверяется отсутствие имён данного запуска в `/dev/shm` и `/dev/mqueue`. Для ожиданий установлены таймауты, а временные файлы создаются вне проекта.

Отдельно выполнен расширенный проверочный прогон из 50 сценариев. Он включал обычные и случайные конфигурации, регистрацию повторного номера, несовпадающие параметры, отсутствие сервера, неверные номера участников, остановку до полного подключения и прекращение работы одного участника. По ленте событий дополнительно проверялся порядок движения и операций с дверями. Все 50 сценариев прошли; эти результаты не суммируются с 24 как число различных тестов, поскольку наборы частично пересекаются.

Для сборок на очередях проверено наличие обращений к `mq_*` и отсутствие `shm_open` и `ftruncate` в таблице внешних символов. Во время работы наблюдались очереди данного сеанса в `/dev/mqueue` и только семафоры, без объекта общей памяти, в `/dev/shm`.

Ниже фактические показатели одного из проверочных прогонов. Все времена в миллисекундах; «ожидание» — среднее время от регистрации появления до подтверждения завершения посадки.

Набор	**Реализация**	**Дост.**	**Время**	**Ожидание**
basic	неименованные	4/4	2628	905,00
basic	именованные	4/4	2632	899,00
basic	независимые, память	4/4	2593	898,75
basic	независимые, очереди	4/4	2616	897,00
group	неименованные	5/5	5056	2332,60
group	именованные	5/5	5059	2334,40
group	независимые, память	5/5	5014	2310,00
group	независимые, очереди	5/5	5025	2307,80
nearest	неименованные	3/3	934	416,00
nearest	именованные	3/3	934	413,33
nearest	независимые, память	3/3	901	403,00
nearest	независимые, очереди	3/3	752	261,33

Эти значения являются примерами корректного выполнения, а не доказательством преимущества одного IPC-механизма. Порядок событий зависит от планировщика. Особенно это заметно при нулевых интервалах появления: первым может зарегистрироваться другой пассажир, что меняет состав первого рейса. Одинаковый `seed` воспроизводит входные интервалы при той же реализации генератора libc, но не весь порядок межпроцессных событий.

10 Использование искусственного интеллекта

Задание я выполнял с ИИ-ассистентом Codex (OpenAI). Работали в диалоге: я задавал вопросы, уточнял требования и просил объяснять механизмы на небольших примерах C. Ассистент объяснял материал, задавал вопросы на понимание, помогал проектировать решение и по моим подтверждениям записывал код отдельных этапов. С его помощью также подготовлены сценарии, скрипт проверок и документация.

На мне были постановка задачи, ограничения по сложности и объёму, изучение предложенных механизмов, подтверждение переходов между этапами и ручная проверка первых сценариев. В частности, я запускал программу в Linux-контейнере, проверял групповую посадку, высадку на нужных этажах, Ctrl+C и отсутствие оставшихся процессов и объектов IPC. Затем по моему разрешению ассистент выполнил автоматические прогоны.

Запросы и ход работы, по которым появлялось решение:

1. **План и изучение материала.** «составь план по смыслу и темам, для каждой объясняй необходимый материал и помогай в написании кода, переходи к следующей после подтверждения» — фрагмент запроса о порядке работы. Отдельно я просил писать простой C, понятный начинающему, и сохранять все обязательные реализации.
2. **Объяснение на примерах.** «не понял эту тему. что за разделяемая память и зачем нужна? приводи примеры». Затем я уточнял, как обозначить такую область в коде и как лифту дождаться готовой команды. Так были разобраны обычные переменные после `fork`, указатель на отображение, `sem_wait`, `sem_post` и порядок копирования сообщения. Я отвечал на разные вопросы по разобранным темам.
3. **Помощь с кодом по этапам.** После обсуждения механизма я разрешал реализацию очередной части. Первый такой запрос: «хорошо. пиши код для первого коммита прямо в файлах, но ничего не коммить, только код».
4. **Проверка и оформление.** «проверь код на наличие ошибок, если что-то не так - пиши»; «делаем отчет и readme. за основу возьми подобные файлы из IHW-2» Ассистент проверял сборку, завершение и освобождение ресурсов. За основу оформления документов взяты README и LaTeX-отчёт предыдущей работы.

11 Критическая оценка сгенерированного кода

1. Первоначальные оценки объёма оказались оптимистичными, поэтому полноту проверяли по требованиям, а не по числу строк. Все варианты сохранены; общие исходники исключают четыре копии модели. Фактический объём C и заголовков — 1262 строки с комментариями и пустыми строками.
2. Длительный `nanosleep` заменён отрезками до 100 мс, а ожидание семафора — короткими таймаутами. Это учитывает сигнал, пришедший между проверкой флага и началом ожидания. Изменения выполнены до основного тестирования.
3. В независимых версиях добавлены регистрация и общий `START`: новый процесс не наследует параметры и время старта диспетчера так, как ребёнок после `fork`.
4. Ответ о регистрации в версии с очередями сделан персональным по PID: отказ дубликату не должен получить действующий участник. Сценарий повторного номера проверен отдельно.
5. Успешные тесты не доказывают корректность при любом планировании и не дают оснований сравнивать быстродействие IPC. Очистка после SIGKILL не гарантируется.

12 Итог

Реализованы четыре варианта одной модели с обоими видами семафоров, общей памятью и очередями POSIX. Поддерживаются журнал, статистика, прерывание и очистка ресурсов. Проверки воспроизводятся командой `make check`, запуск описан в `README.md`.